Blocco codice 1:

```
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

class BaselineRecommender:
    def __init__(self, lambda_reg=10.0, n_iters=15, clip=True):
        """Parameters
        ----------
        lambda_reg : float
            Mantenuto per compatibilità con la firma originale.
        n_iters : int
            Mantenuto per compatibilità con la firma originale.
        clip : bool
            If True, clip predictions to the MovieLens rating scale [1, 5].
        """
        self.lambda_reg = lambda_reg
        self.n_iters = n_iters
        self.clip = clip
        self.fitted = False

        # Inizializzazione delle variabili dei modelli
        self.global_mean = 0.0
        self.user_means = pd.Series(dtype=float)
        self.item_means = pd.Series(dtype=float)
        self.user_bias = pd.Series(dtype=float)
        self.item_bias = pd.Series(dtype=float)

    def fit(self, train):
        """Fit all four baselines using a DataFrame of observed ratings in a
simple, direct way."""
        columns = {"user_id", "movie_id", "rating"}

        if not columns.issubset(train.columns):
            raise ValueError(f"train must contain columns: {sorted(columns)}")
        if train.empty:
            raise ValueError("train must not be empty")

        train = train[["user_id", "movie_id", "rating"]].copy()

        # 1. Global mean (mu)
```

```
        self.global_mean = train["rating"].mean()

        # 2. User mean and 3. Item mean
        self.user_means = train.groupby("user_id")["rating"].mean()
        self.item_means = train.groupby("movie_id")["rating"].mean()

        # 4. User + Item Bias calcolati in modo semplice e diretto (senza cicli
iterativi pesanti)
        # b_i = media dei voti del film - media globale
        item_diffs = train.groupby("movie_id")["rating"].mean() - self.global_mean
        self.item_bias = item_diffs

        # Isoliamo l'effetto del film per calcolare correttamente il bias
dell'utente
        train_copy = train.copy()
        train_copy["item_effect"] =
train_copy["movie_id"].map(self.item_bias).fillna(0.0)

        # Residuo: voto - mu - b_i
        train_copy["residual"] = train_copy["rating"] - self.global_mean -
train_copy["item_effect"]

        # b_u = media dei residui per utente
        user_diffs = train_copy.groupby("user_id")["residual"].mean()
        self.user_bias = user_diffs

        self.fitted = True
        return self
```

Al fine di stabilire un punto di partenza (*benchmark*) rigoroso per i sistemi di raccomandazione, la classe `BaselineRecommender` implementa 4 modelli euristici diversi di predizione basati su statistiche descrittive del dataset *MovieLens 100K*:

- Global Mean ($\hat{r}_{u,i} = \mu$) è la previsione costante basata sulla media globale dei voti ;
- User Mean ($\hat{r}_{u,i} = \bar{r}_u$) è la media dei voti dell'utente u, cattura la tendenza personale dell'utente);
- Item Mean ($\hat{r}_{u,i} = \bar{r}_i$) è la media dei voti per il film i, cattura la qualità intrinseca o la popolarità dell'oggetto);
- User-Item Bias ($\hat{r}_{u,i} = \mu + b_u + b_i$), un modello additivo che scompone il voto combinando la media globale, il bias del film e dell'utente.

Il **costruttore** (`__init__`) predispone i parametri di configurazione (tra cui l'opzione di *clipping* per vincolare i voti alla scala ufficiale 1-5 di MovieLens) e inizializza lo stato booleano `self.fitted = False`, una misura di sicurezza che impedisce l'esecuzione di predizioni prima che il modello sia stato effettivamente addestrato.

Fase di addestramento (fit) → Il codice verifica preventivamente la presenza delle colonne essenziali (`user_id`, `movie_id`, `rating`) e si accerta che il dataset non sia vuoto, sollevando eccezioni mirate in caso di anomalie.
Viene calcolata la **media aritmetica di tutti i voti** presenti nel training set (`self.global_mean`), la quale funge da baseline assoluta in assenza di informazioni specifiche. Vengono calcolate separatamente le **medie per utente** (`self.user_means`) e **per film** (`self.item_means`) tramite operazioni di raggruppamento (`groupby`).

Per quanto riguarda il modello additivo (`user_item_bias`) , l'obiettivo è scomporre ogni voto considerando tre elementi chiave: la tendenza generale della piattaforma, la qualità intrinseca del film e la generosità dell'utente. Il codice calcola dapprima il **bias del film (b_i)**, misurando quanto la media dei voti di una specifica pellicola si discosta dalla media globale ($b_i = \bar{r}_i - \mu$).Successivamente, per calcolare il **bias dell'utente (b_u)** senza distorsioni, il modello sottrae dal voto reale sia la media generale che l'effetto del film appena trovato, isolando così un residuo.

La media di questi residui per ciascun utente definisce direttamente la sua tendenza personale a votare sopra o sotto la media, completando la stima delle preferenze.

Blocco codice 2:

```
    def predict(self, data, method="global_mean"):
        """Predict one rating per row."""
        if not self.fitted:
            raise RuntimeError("Call fit(train) before predict().")
        users = data["user_id"]
        items = data["movie_id"]
```

```
if method == "global_mean":
    predictions = np.full(len(data), self.global_mean)

elif method == "user_mean":
    predictions = (
        users.map(self.user_means)
        .fillna(self.global_mean)
        .to_numpy()
    )

elif method == "item_mean":
    predictions = (
        items.map(self.item_means)
        .fillna(self.global_mean)
        .to_numpy()
    )
elif method == "user_item_bias":
    user_effect = users.map(self.user_bias).fillna(0.0).to_numpy()
    item_effect = items.map(self.item_bias).fillna(0.0).to_numpy()

    predictions = self.global_mean + user_effect + item_effect
else:
    raise ValueError(
        "Unknown method. Choose among: global_mean, user_mean, "
        "item_mean, user_item_bias"
```

```
            )
        if self.clip:
            predictions = np.clip(predictions, 1.0, 5.0)
        return predictions


    def evaluate(self, data):
        """Compute RMSE and MAE for each baseline."""
        if "rating" not in data.columns:
            raise ValueError("data must contain the rating column")
        if data.empty:
            raise ValueError("data must not be empty")

        y_true = data["rating"].to_numpy(dtype=float)
        results = []

        for method in (
            "global_mean",
            "user_mean",
            "item_mean",
            "user_item_bias"
        ):
            y_pred = self.predict(data, method=method)

            rmse = np.sqrt(np.mean((y_true - y_pred) ** 2))
            mae = np.mean(np.abs(y_true - y_pred))
```

```
        results.append({
            "model": method,
            "RMSE": rmse,
            "MAE": mae
        })

    return (
        pd.DataFrame(results)
        .sort_values("RMSE")
        .reset_index(drop=True)
    )
```

Metodo di predizione (predict) → riceve un set di dati e genera le stime dei voti in base alla strategia specificata tramite il parametro `method`. Prima di procedere, il codice verifica che il modello sia stato effettivamente addestrato (`self.fitted`), sollevando un'eccezione in caso contrario.

Per il metodo `global_mean`, viene restituito un array costante contenente la media globale (`self.global_mean`). Per `user_mean` e `item_mean`, le predizioni vengono calcolate mappando gli identificativi degli utenti o dei film sui rispettivi valori medi precedentemente memorizzati. Nel caso in cui si incontri un utente o un film non presente nel training set ***(cold-start)***, il metodo utilizza `.fillna(self.global_mean)` come strategia di riserva ***(fallback)***.

Infine per il modello `user_item_bias`, il calcolo unisce la media globale e i rispettivi effetti di utente e film mappati, gestendo eventuali valori mancanti con uno scostamento nullo (`fillna(0.0)`).

Metodo di Valutazione (evaluate) → Per confrontare l'efficacia di tutti i modelli implementati, il metodo `evaluate` calcola le metriche di errore su un dataset di riscontro (validation o test).

Per ciascuno dei quattro approcci, il codice calcola l'**RMSE** (*Root Mean Squared Error*) e il **MAE** (*Mean Absolute Error*), confrontando i valori reali (`y_true`) con quelli stimati (`y_pred`).

Blocco codice 3

```
    def print_matrix_sparsity(ratings, train, val, test):
        """Compute matrix density and sparsity for each dataset split."""
        n_users = ratings["user_id"].nunique()
        n_movies = ratings["movie_id"].nunique()

        # Total number of possible user-movie combinations
        n_possible = n_users * n_movies
          print(f"\n===  MATRIX  SPARSITY  ({n_users}  users  x  {n_movies}  movies)
===")
        print(f"Possible user-movie pairs: {n_possible:,}")

        for name, subset in (
            ("Full dataset", ratings),
            ("Training", train),
            ("Validation", val),
            ("Test", test),
        ):
            # Count observed user-movie pairs
            n_observed = len(
                subset[["user_id", "movie_id"]].drop_duplicates()
            )
            # Density = observed entries / total possible entries
```

```
        density = n_observed / n_possible

        # Sparsity = missing entries / total possible entries
        sparsity = 1.0 - density
        print(
            f"{name:<13} | Observed: {n_observed:>7,} "
            f"| Density: {density:>7.2%} "
            f"| Sparsity: {sparsity:>7.2%}"
        )
def test_baselines(train, val):
    """Train and compare all four baselines on the validation set."""

    model = BaselineRecommender(lambda_reg=10, n_iters=15)
    model.fit(train)

    methods = [
        "global_mean",
        "user_mean",
        "item_mean",
        "user_item_bias"
    ]

    y_true = val["rating"].to_numpy()
    results = []
```

```
for method in methods:
    # Predict ratings
    y_pred = model.predict(val, method=method)
    y_rounded = np.rint(y_pred)

    # Regression metrics
    rmse = np.sqrt(np.mean((y_true - y_pred) ** 2))
    mae = np.mean(np.abs(y_true - y_pred))

    r2 = 1 - np.sum((y_true - y_pred) ** 2) / np.sum(
        (y_true - y_true.mean()) ** 2
    )

    # Classification-like metrics
    accuracy = np.mean(y_rounded == y_true)
    within_one = np.mean(np.abs(y_rounded - y_true) <= 1)

    results.append({
        "Model": method,
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2,
        "Accuracy": accuracy,
        "Within_1": within_one
    })
```

```
results = pd.DataFrame(results).sort_values("RMSE")


print("\n=== BASELINE COMPARISON ===")

print(results.round(4).to_string(index=False))


# Select the best baseline

best_method = results.iloc[0]["Model"]


# Generate example predictions

examples = val.sample(

    n=min(5, len(val)),

    random_state=42

).copy()


examples["predicted_rating"] = model.predict(

    examples,

    method=best_method

).round(2)


print(f"\n=== PREDICTIONS ({best_method}) ===")

print(

    examples[

        ["user_id", "movie_id", "rating", "predicted_rating"]

    ].to_string(index=False)
```

```
    )

    return model, results, examples
```

Densità e sparsità dei dati: Nei sistemi di raccomandazione, le interazioni tra utenti e oggetti (i voti) formano una *matrice immensa ma estremamente vuota*, poiché ciascun utente recensisce solo una minima frazione dei film disponibili.

La funzione (`print_matrix_sparsity`) calcola il numero totale di combinazioni possibili tra utenti e film (`n_users * n_movies`) e lo confronta con le interazioni effettivamente osservate (`n_observed`), sia sull'intero dataset che sui singoli split (Training, Validation e Test). I valori di ***densità* (percentuale di celle piene)** e ***sparsità* (percentuale di celle vuote**, che in MovieLens 100K supera tipicamente il 93-95%) offrono una prova quantitativa della complessità del problema e della sfida affrontata dagli algoritmi predittivi.

Test e confronto delle Baseline: Per ciascun modello, la funzione `test_baselines` non si limita a calcolare le metriche di regressione standard come l'RMSE e il MAE (che misurano l'errore numerico sulla scala continua dei voti), ma introduce anche metriche avanzate:

- R^2 il coefficiente di determinazione per valutare quanta varianza dei voti il modello riesce a spiegare.
- l'Accuracy (arrotondando le predizioni a interi con np.rint) e l'indice Within_1 (la percentuale di stime che si discostano dal voto reale per al massimo ± 1 stella), molto utili per un'interpretazione pratica dell'errore.

Estrazione campione casuale: Infine, il codice individua automaticamente la strategia migliore basandosi sull'**RMSE minimo**, estrae un **campione casuale di recensioni dal validation set** e stampa una tabella comparativa affiancando il voto reale del fruitore e quello predetto dal modello migliore (arrotondato a due decimali), garantendo una trasparenza totale sui risultati ottenuti.

Blocco di esecuzione principale (__main__):

```
if __name__ == "__main__":

    # Load MovieLens dataset

    ratings = pd.read_csv(
```

```
    "u.data",
    sep="\t",
    names=["user_id", "movie_id", "rating", "timestamp"]
)

# Split: 80% train, 10% validation, 10% test
development, test = train_test_split(
    ratings,
    test_size=0.10,
    stratify=ratings["user_id"],
    random_state=42
)
train, val = train_test_split(
    development,
    test_size=1/9,
    stratify=development["user_id"],
    random_state=42
)
# Ensure all movies are represented in training (cold-start)
missing_movies = (
    set(ratings["movie_id"]) - set(train["movie_id"])
)
for movie in sorted(missing_movies):
    if (val["movie_id"] == movie).any():
        chosen = val[val["movie_id"] == movie].sample(
```

```
            n=1,
            random_state=42
        )
        val = val.drop(chosen.index)

    else:
        chosen = test[test["movie_id"] == movie].sample(
            n=1,
            random_state=42
        )
        test = test.drop(chosen.index)

    train = pd.concat([train, chosen])
# Dataset summary
print("\n=== DATASET SPLIT ===")
print(f"Training:   {len(train)}")
print(f"Validation: {len(val)}")
print(f"Test:       {len(test)}")
# Compute matrix sparsity
print_matrix_sparsity(ratings, train, val, test)
# Run baseline evaluation
model, results, examples = test_baselines(train, val)
```

Il dataset *MovieLens 100K* viene letto tramite Pandas dal file tabulare `u.data`, assegnando esplicitamente le colonne standard (`user_id`, `movie_id`, `rating`, `timestamp`). Per garantire una valutazione robusta evitando fenomeni di *overfitting*

sui dati di test, il dataset viene suddiviso in tre sottoinsiemi distinti: Training, Validation e Test.

Strategia di Stratificazione: La funzione `train_test_split` viene utilizzata applicando l'argomento `stratify=ratings["user_id"]` (dove applicabile). Questo assicura che la distribuzione delle interazioni per utente rimanga omogenea in tutti gli split, evitando che alcuni utenti vengano rappresentati in modo anomalo o sbilanciato. Il calcolo `test_size=1/9` applicato al dataset di sviluppo (`development`) consente di ottenere esattamente il bilanciamento desiderato dell'80% per il training, 10% per il validation e 10% per il test.

Gestione del problema di cold-start: nei sistemi di raccomandazione basati su interazioni storiche, un modello non è in grado di elaborare stime affidabili qualora compaia un utente o un oggetto del tutto nuovo, privo di qualsiasi storico (*cold start* puro).

Lo script esegue un controllo preliminare confrontando gli identificativi dei film presenti nel dataset completo con quelli presenti nel training set (`missing_movies`). Per ogni film mancante nel training, il codice individua un'istanza corrispondente nel validation set (o, in subordine, nel test set), la rimuove dallo split di riscontro e la sposta dinamicamente all'interno del training set tramite `pd.concat`.

Questa scelta metodologica garantisce che il modello venga testato esclusivamente su scenari in cui possiede un profilo informativo sufficiente, assicurando che l'errore di predizione (RMSE) sia metricamente coerente con i limiti strutturali delle baseline e dei futuri modelli a fattori latenti.

Analisi dei risultati →

```
=== DATASET SPLIT ===

Training:   80021

Validation: 9990

Test:       9989

=== MATRIX SPARSITY (943 users x 1682 movies) ===

Possible user-movie pairs: 1,586,126

Full dataset  | Observed: 100,000 | Density:   6.30% | Sparsity:  93.70%

Training      | Observed:  80,021 | Density:   5.05% | Sparsity:  94.95%

Validation    | Observed:   9,990 | Density:   0.63% | Sparsity:  99.37%
```

```
Test          | Observed:   9,989 | Density:   0.63% | Sparsity:  99.37%


=== BASELINE COMPARISON ===

         Model   RMSE    MAE      R2  Accuracy  Within_1
user_item_bias 0.9432 0.7431  0.2965    0.4159    0.8901
     item_mean 1.0294 0.8198  0.1620    0.3657    0.8624
     user_mean 1.0357 0.8263  0.1516    0.3738    0.8550
   global_mean 1.1245 0.9432 -0.0001    0.3412    0.8234


=== PREDICTIONS (user_item_bias) ===

 user_id  movie_id  rating  predicted_rating
     145      1002       1              2.29
     454       124       4              3.37
      90       966       5              4.56
     191       754       3              3.44
     798       768       4              3.20
```

Sparsità → Con 943 utenti e 1.682 film, lo spazio totale delle interazioni possibili ammonta a 1.586.126 coppie. L'intero dataset contiene 100.000 valutazioni effettive, confermando una **sparsità complessiva del 93,70%** (densità del 6,30%).

Baseline → Global Mean ottiene le **prestazioni peggiori** (RMSE = 1.1245, $R^2 \approx 0$), confermando che una stima costante basata unicamente sulla media generale non è in grado di catturare le specificità delle preferenze individuali o dei film.

User Mean e Item Mean registrano performance simili tra loro, dimostrativi del fatto che considerare singolarmente la tendenza dell'utente o la popolarità dell'oggetto riduce significativamente l'errore rispetto alla media globale (R^2 che sale attorno al 15-16%).

User-Item Bias si conferma il modello migliore raggiungendo un RMSE di 0.9432 e un MAE di 0.7431, con un coefficiente R^2 quasi raddoppiato (0.2965). Questo dimostra che la scomposizione additiva lineare ($b_u + b_i$) cattura efficacemente sia la generosità/severità dell'utente che la qualità intrinseca del film. L'indice *Within_1* del modello `user_item_bias` raggiunge l'89,01%, indicando che quasi nove previsioni su

dieci si discostano dal voto reale per al massimo una stella (con un'Accuracy del 41,59% sui voti esatti).

In conclusione, **un RMSE di 0.9432 sul validation set rappresenta un solido benchmark iniziale** pronto per essere superato dalle future milestone basate su modelli di fattorizzazione o collaborative filtering avanzato.